

Key Takeaways

1 The bottleneck problem limits fixed-size encoders

Without attention, the encoder must compress ALL source information into a single fixed-size vector. For long sentences, this is a fundamental information bottleneck — semantic content, word order, and grammar all compete for the same limited dimensions. Performance degrades sharply for sentences longer than ~ 20 words.

2 Bahdanau attention computes dynamic context vectors

At each decoder step t , alignment scores $e_{ti} = v^\top \tanh(W_1 s_{t-1} + W_2 h_i)$ measure relevance of each source position. Softmax normalises these to attention weights α_{ti} . The context vector $c_t = \sum_i \alpha_{ti} h_i$ aggregates source information relevant to the current output word.

3 Teacher forcing accelerates training but creates exposure bias

During training, we feed *ground-truth* tokens to the decoder at probability 0.5, causing faster convergence. However, at test time the decoder sees its own (possibly wrong) predictions — a train/test mismatch called **exposure bias**. Curriculum learning gradually reduces teacher forcing from 1.0 to 0.0 to bridge this gap.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1. Explain the “information bottleneck” in seq2seq models without attention. Why does performance degrade on longer sentences?
- Q2. In Bahdanau attention, the score function is $e_{ti} = v^\top \tanh(W_1 s_{t-1} + W_2 h_i)$. What does a high score e_{ti} mean in terms of the translation process?
- Q3. What is teacher forcing, and why does it create “exposure bias”? How does curriculum learning address this problem?
- Q4. An attention heatmap shows near-diagonal structure for English→French translation. What does this reveal about word order alignment between the two languages?
- Q5. A model achieves BLEU= 0.15. Is this good, bad, or context-dependent? Justify your answer.

Coding Exercises

Work through these in a Colab notebook with GPU enabled.

Exercise 1: Train and Visualize Attention Heatmaps

Train the full Seq2Seq + Attention model on the Tatoeba dataset (filtered to ≤ 10 words, $\sim 10K$ pairs). After training, translate 5 test sentences. For each translation, plot an attention heatmap: English words on the x-axis, French output words on the y-axis, with color intensity showing attention weight. Mark the highest-weight source position for each output token. *Expected output:* 5 attention heatmaps showing learned word alignment.

Exercise 2: Ablation — Seq2Seq Without Attention

Remove the attention module. Train a plain encoder-decoder where the decoder receives only the final encoder hidden state. Train for the same number of epochs with identical hyperparameters. Compare: (a) BLEU scores on the test set, (b) example translations for 5 sentences, (c) qualitative failure cases. Which sentence patterns break first without attention? *Expected output:* BLEU comparison bar chart + example translation table.

Exercise 3: Luong vs Bahdanau Attention

Replace Bahdanau's additive attention score $e_{ti} = v^\top \tanh(W_1 s_{t-1} + W_2 h_i)$ with Luong's dot-product score $e_{ti} = s_t^\top h_i$ (requires matching encoder/decoder hidden sizes). Train both variants and compare: BLEU scores, training speed (time per epoch), and attention heatmaps on the same 3 test sentences. *Expected output:* BLEU comparison + speed comparison + side-by-side attention heatmaps.

Challenge Problem

Implement Beam Search Decoding (beam width = 3):

1. Encode the source sentence to get encoder hidden states
2. Initialise 3 beams, each starting with <SOS> and log-probability = 0
3. At each step: expand each beam over the full vocabulary, compute $\log p(\text{next token}|\text{beam so far})$
4. Keep only the top-3 beams by cumulative log-probability
5. Stop a beam when it generates <EOS> or reaches `max_length`
6. Compare greedy vs beam search BLEU on 50 test sentences

Bonus: Add length normalisation — divide cumulative log-probability by output length to prevent beam search from preferring shorter translations.

Further Reading

- **Attention Mechanism:** Bahdanau, Cho & Bengio (2015), “*Neural Machine Translation by Jointly Learning to Align and Translate*” — the paper that introduced attention; Sections 2–3 are essential reading.
- **Attention Variants:** Luong, Pham & Manning (2015), “*Effective Approaches to Attention-based NMT*” — global vs. local attention, dot-product vs. additive scoring.
- **Original Seq2Seq:** Sutskever, Vinyals & Le (2014), “*Sequence to Sequence Learning with Neural Networks*” — introduced the encoder-decoder architecture.
- **BLEU Metric:** Papineni et al. (2002), “*BLEU: a Method for Automatic Evaluation of Machine Translation*” — understand the n-gram precision formula and its limitations.

Online Resources & Tools

- **PyTorch Seq2Seq tutorial** — official NMT guide
- **SacreBLEU** — standardised BLEU evaluation
- **Fairseq** — Facebook translation toolkit
- **Jay Alammar’s blog** — Illustrated Seq2Seq
- **NLTK** — `nltk.translate.bleu_score`
- **HuggingFace datasets** — parallel corpora

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Confused about bidirectional GRUs?** Week 6 Session 1 — a bidirectional GRU reads left-to-right AND right-to-left, giving each position context from both directions. The forward and backward hidden states are concatenated, doubling the hidden dimension.
- **Unsure about packed sequences?** `torch.nn.utils.rnn.pack_padded_sequence` tells the RNN to skip PAD positions during computation — required for efficient batching of variable-length sentences.
- **Forgot GRU vs LSTM?** GRU has only 2 gates (reset gate r , update gate z) vs. LSTM’s 3. GRU is faster (fewer parameters) with comparable quality on most tasks. Session 4 uses GRU in the encoder for speed.
- **Unsure about word embeddings?** They are the same `nn.Embedding` from Session 3, but now we have two separate embedding tables: one for English words and one for French words.

Answers — Test Your Understanding

- A1.** Without attention, the encoder must compress the ENTIRE source sentence into a single fixed-size vector (e.g., 256-dim). A 20-word sentence contains far more semantic information than a 256-dim vector can reliably encode. Empirically, BLEU drops sharply for sentences > 20 words as critical information is overwritten during encoding.
- A2.** A high e_{ti} means the decoder state s_{t-1} is highly “compatible” with encoder state h_i in the learned attention space. In translation terms: the i -th source word is especially relevant for generating the t -th output word. This is the model’s learned word alignment — “to generate *chat*, attend to *cat*”.
- A3.** Teacher forcing feeds ground-truth previous tokens to the decoder during training, accelerating convergence significantly. Exposure bias occurs because at test time the decoder uses its own (possibly wrong) previous predictions, creating a distribution mismatch. Curriculum learning reduces the teacher forcing ratio from 1.0 to 0.0 over training, gradually exposing the decoder to its own predictions.
- A4.** A near-diagonal heatmap means the i -th English word dominantly attends to the i -th French word — roughly monotonic alignment. English and French share similar word order (SVO), so this is expected. German-English would show crossing patterns due to German’s verb-final ordering.
- A5.** Context-dependent. BLEU = 0.15 is “low quality” by academic standards (human translators score ~ 0.6 , Google Translate ~ 0.4 on standard benchmarks). However, for a small model trained on only 10K pairs for 25 epochs, it indicates the model has learned non-trivial alignment and correctly translates simple patterns. Compare against the no-attention baseline to quantify attention’s contribution.

Deep Dive: Attention as Differentiable Soft Alignment

Bahdanau attention computes a **soft alignment** over all source positions simultaneously. The three steps at each decoder timestep t :

Step 1 — Score: Measure compatibility between decoder state s_{t-1} and each encoder output h_i :

$$e_{ti} = v^\top \tanh(W_1 s_{t-1} + W_2 h_i), \quad i = 1, \dots, T_x$$

Step 2 — Normalise: Convert scores to a probability distribution over source positions:

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_{j=1}^{T_x} \exp(e_{tj})}$$

Step 3 — Aggregate: Compute the context vector as a weighted sum of encoder outputs:

$$c_t = \sum_{i=1}^{T_x} \alpha_{ti} h_i$$

The key insight: unlike “hard” attention (picking one position, requiring RL to train), soft attention is **fully differentiable** — gradients flow through the α weights back to the encoder, allowing end-to-end training.

Deep Dive: BLEU Score — The Math

BLEU measures n -gram precision between hypothesis and reference translation:

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right)$$

where p_n = modified n -gram precision, $w_n = 1/N$, and BP = brevity penalty.

Worked example: Reference: “the cat is on the mat”. Hypothesis: “the cat sat on the mat”.

- Unigram precision p_1 : 5/6 words match (“sat” is wrong)
- Bigram precision p_2 : “the cat”, “on the”, “the mat” match $\Rightarrow 3/5$
- $BP = 1$ (same length), $\text{BLEU} \approx \exp(0.5 \cdot \log(5/6) + 0.5 \cdot \log(3/5)) \approx 0.54$

Deep Dive: Encoder–Decoder Information Flow

Tensor dimensions for a batch of 64 sentences, max length 10, hidden size 256:

Component	Tensor	Shape
Source embeddings	E_{src}	[64, 10, 256]
Encoder outputs (Bi-GRU)	H	[64, 10, 512] (fwd+bwd concat)
Encoder final hidden	s_0	[64, 256]
Attention scores	e_t	[64, 10] (one score per source pos.)
Attention weights	α_t	[64, 10] (sum to 1)
Context vector	c_t	[64, 512]
Decoder input (embed + context)	d_t	[64, 768]
Decoder hidden	s_t	[64, 256]
Output logits	y_t	[64, tgt_vocab_size]

Systematic Experiments

Record results in a table. Run each experiment independently, changing only the stated variable:

1. **Bidirectional vs. unidirectional encoder:** Replace bidirectional GRU with a unidirectional GRU (same total hidden size). Compare BLEU scores. *Hypothesis:* bidirectional is better for encoding because it sees full context before producing hidden states.
2. **Sentence length filtering:** Train 3 models using datasets filtered to ≤ 5 , ≤ 10 , ≤ 15 words. Plot BLEU as a function of test sentence length. Where does each model degrade?
3. **Teacher forcing ratio:** Train models with teacher forcing ratios $\in \{0, 0.25, 0.5, 0.75, 1.0\}$. Compare final BLEU and training convergence speed. Does higher teacher forcing always help?
4. **Hidden size:** Train with hidden sizes $\in \{64, 128, 256, 512\}$. Plot BLEU vs. training time. Find the knee of the curve where capacity gains diminish.

Modern Seq2Seq Variants

The architecture in Session 4 was SOTA in 2015. What came next:

- **Multi-head attention (Transformer):** h parallel attention heads each attend with different W^Q, W^K, W^V projections. Captures multiple types of alignment simultaneously. Fully replaces RNNs in Session 5.
- **Copy mechanism:** Allows the decoder to directly copy tokens from the source sequence (e.g., proper nouns, numbers). Critical for tasks with many OOV words.
- **Coverage mechanism:** Tracks how much attention each source position has received. Penalises attending to the same word multiple times — prevents over-translation and missing words.
- **Subword tokenisation (BPE/SentencePiece):** Splits rare words into subword units, reducing OOV rate and vocabulary size. Used in all modern NMT systems.

Cross-Lingual Transfer: Zero-Shot Translation

If you train on English→French and English→Spanish pairs simultaneously (sharing encoder weights), can the model translate French→Spanish without ever seeing that pair?

1. Train a multilingual seq2seq model on 3 language pairs: EN→FR, EN→ES, FR→EN
2. Add a language tag token to each target sentence: <fr> or <es>
3. Test on FR→ES (zero-shot): encode a French sentence and decode with tag <es>
4. Measure BLEU on 100 French→Spanish pairs

This is the principle behind mBERT, mT5, and NLLB (No Language Left Behind). The shared encoder forces the model to learn a language-agnostic representation.

Attention Mechanisms Comparison

Type	Score Function	Parameters	When to Use
Bahdanau (additive)	$v^\top \tanh(W_1 s + W_2 h)$	W_1, W_2, v	Default choice, flexible
Luong dot-product	$s^\top h$	None	Fast, requires equal dims
Luong general	$s^\top W h$	W	Flexible, bilinear
Luong concat	$v^\top \tanh(W[s; h])$	W, v	Similar to Bahdanau
Multi-head (Transformer)	$\text{softmax}(QK^\top / \sqrt{d_k})V$	W^Q, W^K, W^V per head	Modern, parallelisable

Translation Quality Metrics

Metric	What It Measures	Range	Limitation
BLEU	N-gram precision	0–1	Penalises valid paraphrases
ROUGE-L	Longest common subsequence	0–1	Recall-oriented
METEOR	Precision + recall + order	0–1	Language-dependent
BERTScore	Contextual embedding similarity	0–1	Requires pre-trained BERT
chrF	Character-level F-score	0–1	Better for morphologically rich languages

Common Translation Errors & Debugging

Error Type	Symptom	Likely Cause & Fix
Hallucination	Output words not related to source	Attention too diffuse; try label smoothing
Repetition	Same word repeats in output	Coverage issue; add coverage loss penalty
Too-short translations	Missing words, brevity penalty	Apply length normalisation in beam search
Wrong word order	Grammatically incorrect output	Increase training epochs; try bidirectional encoder
OOV words	<UNK> tokens in output	Use subword tokenisation (BPE)

Hands-on notebook: `Session_4_Seq2Seq_Attention_Translation.ipynb`

Instructor: Priyansh Saxena